BÁO CÁO TASK JIRA

CNPM-126

[17.3] Cấu hình lưu trữ đám mây và quản lý mã nguồn

Triển khai minh họa bằng Python và Firebase Cloud Storage

Mã công việc	CNPM-126
Hạng mục	Cloud Storage và quản lý source code
Công nghệ	Python, Firebase Admin SDK, Git, GitHub
Ngôn ngữ	Python — không sử dụng C++

Báo cáo gồm mô tả giải pháp, cài đặt, thao tác, bảo mật, kiểm thử và phần mã nguồn Python. Cần Firebase project và credential hợp lệ để chạy trên môi trường thật.

1. Giới thiệu

Task CNPM-126 yêu cầu cấu hình lưu trữ đám mây và quản lý mã nguồn dự án. Tài liệu chọn Firebase Cloud Storage làm dịch vụ lưu trữ; Python làm ngôn ngữ triển khai các thao tác backend/CLI; Git và GitHub dùng để quản lý phiên bản.

Mục tiêu

- Kết nối Python với Firebase Cloud Storage.
- Tải lên, tải xuống, liệt kê và xóa tệp lưu trữ.
- Thiết lập quy tắc truy cập mẫu, bảo vệ credential.
- Hướng dẫn đẩy source code lên GitHub và gắn tag Sprint.

2. Công nghệ

Công nghệ	Vai trò
Python 3.10+	Viết CLI thao tác Storage
Firebase Admin SDK	Khởi tạo Firebase và bucket
Google Cloud Storage	Nơi lưu trữ đối tượng
python-dotenv	Đọc cấu hình từ file .env
Git/GitHub	Quản lý lịch sử và tag phiên bản

3. Chức năng và luồng xử lý

Luồng: nhập lệnh CLI → đọc cấu hình .env → xác thực service account → thao tác Firebase bucket → hiển thị kết quả.

- Upload: tải tệp máy local lên bucket.
- Download: tải object về máy.
- List: liệt kê object, hỗ trợ lọc prefix.
- Delete: xóa object sau khi xác nhận DELETE.

4. Cài đặt và cấu hình

Yêu cầu: Python 3.10+, Firebase project đã bật Cloud Storage, service account đủ quyền và Git cài đặt.

Tạo virtual environment, cài dependencies

```
python -m venv .venv\n# Windows: .venv\\Scripts\\activate\n# macOS/Linux: source .venv/bin/activate\npip install -r requirements.txt\ncopy .env.example .env
```

Thiết lập file .env

```
FIREBASE_STORAGE_BUCKET=your-project-id.appspot.com\nGOOGLE_APPLICATION_CREDENTIALS=./service-account.json
```

Điền bucket name và đường dẫn JSON service account theo Firebase project. Tuyệt đối không commit file credential lên GitHub.

5. Các lệnh sử dụng

```
python app.py upload ./sample.txt documents/sample.txt\npython app.py list\npython app.py list --prefix documents/\npython app.py download documents/sample.txt
```

Lệnh delete hỏi người dùng nhập DELETE. Nếu xác nhận không đúng, chương trình sẽ hủy thao tác.

6. Bảo mật

Tệp storage.rules mẫu cho phép đọc/ghi đối với người dùng đã đăng nhập. Triển khai bằng Firebase CLI:

```
firebase deploy --only storage
```

- .env và service-account.json đã được đưa vào .gitignore.
- Chỉ cấp quyền IAM tối thiểu cho service account.
- Firebase Admin SDK sử dụng quyền service account, không chịu Storage Rules theo cách client SDK chịu; phải bảo vệ khóa.
- Ứng dụng production cần bổ sung phân quyền theo vai trò/owner.

7. Quản lý mã nguồn GitHub

```
git init
git add .
git commit -m "CNPM-126: configure Firebase Storage with Python"
git branch -M main
git remote add origin https://github.com/<username>/<repository>.git
git push -u origin main
git tag -a sprint-3 -m "CNPM-126 Sprint 3"
git push origin sprint-3
```

Kiểm tra credential trước khi push:

```
git status\ngit check-ignore .env service-account.json
```

Tag sprint-3 chỉ là ví dụ, cần đổi theo quy ước Sprint của nhóm.

8. Kịch bản kiểm thử

STT	Kiểm thử	Kết quả mong đợi
1	Upload tệp hợp lệ	Object xuất hiện trên bucket
2	List object	Tên và kích thước hiển thị
3	Download object	Tệp được lưu về đúng đường dẫn
4	Delete, nhập DELETE	Object được xóa
5	Delete, không xác nhận	Thao tác bị hủy
6	Thiếu cấu hình	Hiển thị thông báo lỗi

Các dòng trên là kịch bản đề xuất; cần chạy thực tế để xác nhận kết quả.

9. Kết luận

Project Python cung cấp các thao tác cơ bản với Firebase Cloud Storage, đi kèm hướng dẫn bảo vệ credential và quản lý source code trên GitHub. Khi triển khai thực tế, cần cấu hình IAM, bổ sung phân quyền người dùng và kiểm thử trên Firebase project của nhóm.

PHỤ LỤC — MÃ NGUỒN PYTHON

Mã nguồn app.py từ project CNPM-126 được đính kèm ở phần dưới.

```
"""
CNPM-126: Cloud storage configuration and source-code management.
Python CLI using Firebase Admin SDK / Google Cloud Storage.
"""
import argparse
import os
from pathlib import Path

import firebase_admin
from firebase_admin import credentials, storage
from dotenv import load_dotenv

load_dotenv()


def initialize_firebase():
    bucket_name = os.getenv("FIREBASE_STORAGE_BUCKET")
    credentials_path = os.getenv("GOOGLE_APPLICATION_CREDENTIALS")

    if not bucket_name:
        raise RuntimeError("Missing FIREBASE_STORAGE_BUCKET in .env")
    if not credentials_path or not Path(credentials_path).is_file():
        raise RuntimeError(
            "Set GOOGLE_APPLICATION_CREDENTIALS to the path of your service-account JSON."
        )

    if not firebase_admin._apps:
        cred = credentials.Certificate(credentials_path)
        firebase_admin.initialize_app(cred, {"storageBucket": bucket_name})
    return storage.bucket()


def safe_object_name(name: str) -> str:
    """Normalize object path and reject traversal-like names."""
    normalized = name.replace("\\", "/").strip("/")
    if not normalized or any(part == ".." for part in normalized.split("/")):
        raise ValueError("Invalid storage object path.")
    return normalized


def upload_file(local_path: str, object_name: str):
    bucket = initialize_firebase()
    path = Path(local_path)
    if not path.is_file():
        raise FileNotFoundError(f"Local file not found: {path}")
    blob = bucket.blob(safe_object_name(object_name))
    blob.upload_from_filename(str(path))
    print(f"Uploaded: {path} -> gs://{bucket.name}/{blob.name}")
```

```
def download_file(object_name: str, destination: str):
    bucket = initialize_firebase()
    blob = bucket.blob(safe_object_name(object_name))
    if not blob.exists():
        raise FileNotFoundError(f"Storage object not found: {object_name}")
    dest = Path(destination)
    dest.parent.mkdir(parents=True, exist_ok=True)
    blob.download_to_filename(str(dest))
    print(f"Downloaded: gs://{bucket.name}/{blob.name} -> {dest}")


def list_files(prefix: str = ""):
    bucket = initialize_firebase()
    safe_prefix = safe_object_name(prefix) if prefix.strip("/") else ""
    found = False
    for blob in bucket.list_blobs(prefix=safe_prefix):
        found = True
        print(f"{blob.name} ({blob.size} bytes)")
    if not found:
        print("No objects found.")


def delete_file(object_name: str):
    bucket = initialize_firebase()
    blob = bucket.blob(safe_object_name(object_name))
    if not blob.exists():
        raise FileNotFoundError(f"Storage object not found: {object_name}")
    confirm = input(f"Delete '{object_name}' permanently? Type DELETE to confirm: ")
    if confirm != "DELETE":
        print("Cancelled.")
        return
    blob.delete()
    print(f"Deleted: {object_name}")


def main():
    parser = argparse.ArgumentParser(description="CNPM-126 private cloud storage CLI")
    sub = parser.add_subparsers(dest="command", required=True)

    upload = sub.add_parser("upload", help="Upload a local file")
    upload.add_argument("local_path")
    upload.add_argument("object_name")

    download = sub.add_parser("download", help="Download a storage object")
    download.add_argument("object_name")
    download.add_argument("destination")
```

```
    listing = sub.add_parser("list", help="List objects")
    listing.add_argument("--prefix", default="")

    delete = sub.add_parser("delete", help="Delete an object (confirmation required)")
    delete.add_argument("object_name")

    args = parser.parse_args()
    if args.command == "upload":
        upload_file(args.local_path, args.object_name)
    elif args.command == "download":
        download_file(args.object_name, args.destination)
    elif args.command == "list":
        list_files(args.prefix)
    elif args.command == "delete":
        delete_file(args.object_name)


if __name__ == "__main__":
    main()
```